

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Extend the program to demonstrate how improper synchronization can lead to zombie processes. Use appropriate system calls to prevent zombie processes.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int i, status;

    // Create 3 child processes
    for (i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] < 0) {
            printf("Fork failed!\n");
            exit(1);
        }

        if (pid[i] == 0) {
            // Child Process
            printf("Child %d started\n", i + 1);
            printf("PID = %d\n", getpid());
            printf("PPID = %d\n", getppid());

            sleep(i + 2); // Different execution times

            printf("Child %d completed\n\n", i + 1);
            exit(0);
        }
    }

    // Parent waits for all children using wait()
    printf("\nParent waiting using wait()\n\n");

    for (i = 0; i < 3; i++) {
        wait(&status);
        printf("A child process terminated.\n");
    }

    printf("\nAll child processes completed using wait().\n");

    return 0;
}
```

```
prajwal@PRAJWAL:~/os$ nano p4.c
prajwal@PRAJWAL:~/os$ gcc p4.c -o p4
prajwal@PRAJWAL:~/os$ ./p4
Child 2 started
Child 1 started
PID  = 643
PPID = 641
PID  = 642
PPID = 641

Parent waiting using wait()

Child 3 started
PID  = 644
PPID = 641
Child 1 completed

A child process terminated.
```